LLM 应用框架横纵分析报告

> 从 LangChain 到 Dify，从 Chain 到 Agent，从"写代码编排"到"拖拽搭建"，LLM 应用框架如何定义了什么叫做"AI 原生应用"。

一、一句话定义

LLM 应用框架是**为构建基于大语言模型的应用程序而设计的开发工具和运行时环境**。它的核心矛盾在于：底层的 LLM API 只提供"一问一答"的能力，但真实的应用需要检索、记忆、工具调用、多步推理等复杂编排——框架在这个"API 的简单性"和"应用的复杂性"之间架起桥梁。

二、技术背景

核心矛盾：LLM API 太简单，应用太复杂

2022 年末 ChatGPT 引爆浪潮后，开发者很快发现一个尴尬的事实：**裸调 OpenAI API 能做演示，但做不了产品。**

一个真实的 AI 应用需要的远不止"发送 prompt → 接收回复"：

- **RAG**：从大量文档中检索相关内容
- **工具调用**：让 LLM 能搜索、计算、查询数据库
- **记忆管理**：多轮对话的上下文保持
- **Agent 循环**：LLM 自主决定下一步做什么
- **评估**：如何判断输出质量

这些能力分散在不同技术栈中，每个都需要大量基础设施代码。LLM 应用框架的出现就是把这些"通用基础设施"标准化，让开发者把精力放在业务逻辑上。

> 前置知识请参考独立文档 《LLM应用框架_前置知识》，涵盖 Agent、Pipeline、Memory、Tool 等基础概念的通俗讲解。

三、纵向分析：从 Chain 到 Agent 的演化

3.1 萌芽期：做第一个"LLM 应用"的人（2022.10–2023.03）

LLM 应用框架的历史起点非常精确——2022 年 10 月，Harrison Chase 发布了 LangChain。

当时的背景：GPT-3.5 刚发布，ChatGPT 尚未推出（2022.11），开发者群体刚刚开始尝试用 LLM API 构建应用。Harrison Chase 当时在机器学习初创公司 Robust Intelligence 工作，他发现每次构建 LLM 应用都要重复写 prompt 模板、上下文管理、工具集成这些代码——为什么不做成一个库?

几乎同时（2022.11），Jerry Liu 发布了 GPT Index（后更名为 LlamaIndex），最初定位极为专注——**解决"怎么把私有数据喂给 LLM"这个具体问题**。当时 GPT-3 的 4K 上下文窗口放不进一份完整的公司文档，LlamaIndex 做的就是文档分块、索引、检索。

这两者在起点的差异埋下了后来分工的种子：**LangChain 做"流程编排"，LlamaIndex 做"数据检索"。**

2023 年 3–4 月是融资爆发期。LangChain Inc. 注册成立，一周内先后获得 Benchmark 的 1000 万美元种子轮和 Sequoia Capital 的 2000 万美元 A 轮，估值至少 2 亿美元。这在当时是一个令人瞠目的估值——LangChain 才发布了 4 个月，没有任何收入。

同期，德国的 **deepset** 公司把 Haystack 从传统 NLP 框架转向 LLM 应用框架——Haystack 早在 2020 年就开源了，最初服务于 BERT 时代的检索和 QA 场景。2023 年的 LLM 浪潮对它来说是一次"老树发新芽"。

Microsoft 也在这时入局，发布 Semantic Kernel —— 一个与 Azure 生态和 M365 Copilot 深度绑定的 AI 编排 SDK。

3.2 扩张期：框架的"圈地运动"（2023–2024）

2023 年是框架快速迭代、疯狂扩展的一年。

LangChain 的 LCEL（LangChain Expression Language）在 2023 年 Q3 推出。LCEL 的宣言式链定义替代了早期命令式的 chain 构建——用 `|` 操作符把组件串在一起。这是一个重要的设计决策：**框架不应该让开发者写大量胶水代码来组合 LLM 能力。**

2023 年 10 月，LangChain 发布 LangServe（部署工具）和 LangGraph（Agent 编排模块）——前者把 LCEL 代码部署为 REST API，后者引入了循环图（Cyclic Graph）来处理 Agent 的"推理→行动→推理"循环。

LangGraph 是 2024–2026 年间 LangChain 最重要的产品决定。 它意识到 DAG（有向无环图）链无法处理 Agent 逻辑——Agent 本质上是一个 LLM for-loop，需要循环能力。StateGraph 模块让开发者定义状态对象，节点返回更新，支持 override 和 append 两种模式。

2024 年 1 月，LangChain v0.1 稳定版发布，标志着 API 从快速迭代期进入稳定期。同月，LlamaIndex 也正式从 GPT Index 更名为 LlamaIndex（去 OpenAI 绑定），发布完整 RAG 管线支持。

Dify 在 2023 年开源发布，走上了完全不同的路——**可视化拖拽**。当 LangChain 和 LlamaIndex 在比谁集成更多、API 更灵活时，Dify 瞄准了一个被忽视的群体：不会写代码的业务团队和产品经理。它的 Prompt IDE、拖拽 Workflow、内置 LLMOps 构成了一套"开箱即用"的完整平台体验。

Dify 的增长曲线从 2023–2024 年就已经开始加速——不是因为它的技术更先进，而是因为 AI 应用的决策者不只是工程师，还包括需要快速验证想法的产品团队和业务负责人。

这个阶段形成了 LLM 应用框架的"三国格局"：

- **LangChain**：编排之王，生态最大，Agent 最强

- **LlamaIndex**：检索之王，文档处理最深
- **Dify**：低代码之王，用户量增长最快（2024 年已接近 5 万 stars）

3.3 Agent 化：所有框架都在"长出 Agent"（2024–2025）

2024–2025 年是框架的"Agent 化"浪潮。**单次问答已经不够了——框架都在向 Agent 的方向演进。**

LangGraph 发布于 2024 年 1 月，成为 LangChain 生态的 Agent 核心。2025 年 5 月，LangGraph Platform GA——提供托管基础设施，用于部署长时间运行、有状态的 Agent。引入 Deep Agents（支持规划、子 Agent、虚拟文件系统）。LangGraph 的 Pregel 式架构（受 Google 图计算模型启发）成为 Agent 编排的事实标准。

Microsoft 的双框架合并：2025–2026 年，微软的 AI 编排策略发生重大变化。Semantic Kernel 正式更名为 **Microsoft Agent Framework（MAF）**。AutoGen（微软研究院的多 Agent 对话框架）进入维护模式，新用户直接使用 MAF，现有用户走迁移指南。这是微软内部的一次框架收敛——不再让两个框架竞争，统一到一个企业级产品。

LlamaIndex 推出了 Workflows（2025）——它的 Agent 编排模块。LlamaCloud/LlamaParse 商业平台包括 LlamaAgents（端到端文档 Agent 构建）。LlamaParse 支持 130+ 格式，到 2026 年处理了超过 10 亿文档。

CrewAI（2024 年发布）走了一条轻量级的多 Agent 协作路线，50k+ stars，独立于 LangChain 实现。它的设计理念是"多 Agent 角色扮演"——每个 Agent 有角色、目标、背景故事。简单得有些"玩具感"，但它的快速增长说明**多 Agent 模式的入门需要比 LangGraph 更低的门槛**。

3.4 平台化与收敛：Dify 超越 LangChain（2025–2026）

2026 年是框架格局剧烈变化的年份。

最令人意外的事件：**Dify 以 140k GitHub Stars 超越 LangChain（135k）**，成为 LLM 应用框架星数第一。这个数字不是巧合——Dify 在 2026 年的关键动作：

- **MCP 协议原生集成**（2026 年 4 月）：同时支持 HTTP-based MCP 服务和将自身工作流发布为 MCP Server
- **企业版 + 私有部署**：被大量 GCC（政府/央企）开发者采用
- **500 万+ 下载量，800+ 贡献者**

LangChain 仍然是最广泛使用的 Agent 编排框架（LangGraph 的 Agent 能力没有被超越），但 **Dify 证明了"低代码 + 自托管"的路线图在 2026 年的 AI 应用市场中占据了不可忽视的位置。**

Haystack 在受监管行业保持了稳定的增长——Apple、Meta、NVIDIA、Databricks、Intel 都是它的企业客户。它的模块化、可审计管线设计在金融、医疗和政府场景上的优势是其他框架难以替代的。

DSPy 走上了完全不同的道路——作为编译器而非框架，自动优化 prompt 和 few-shot 示例。学术认可度高（ICLR 2024 Spotlight），34k stars。

到 2026 年，框架格局基本厘清：

- **Agent 编排**：LangGraph 领先
- **检索/文档处理**：LlamaIndex 领先
- **低代码/快速原型**：Dify 领先
- **监管行业**：Haystack 领先

- **企业 .NET 生态**：MAF（原 Semantic Kernel）领先
- **Prompt 自动优化**：DSPy 独特路线

四、横向分析：竞争图谱

4.1 场景判断

LLM 应用框架是充分竞争市场（场景 C），六个主要框架各自主攻不同方向。

4.2 核心框架对比

维度	LangChain	LlamaIndex	Dify	Haystack	MAF (原SK)	DSPy
发布时间	2022.10	2022.11	2023	2020	2023	2023
创建方	Harrison Chase	Jerry Liu	LangGenius	deepset (Berlin)	Microsoft	Stanford
GitHub Stars	**135k**	49k	**140k**	25k	28k	34k
核心定位	编排+Agent	检索+文档	低代码平台	审计管线	企业.NET	Prompt 编译器
可视化	❌	❌	✅ 拖拽	❌	❌	❌
Agent 能力	**最强** (LangGraph)	中 (Workflows)	强 (FC/ReAct)	中	强(多 Agent)	弱
RAG 灵活度	高	**极高**	高	高	中	通过编译
学习曲线	中–高	中	**低**	中	中–高	高
商业模式	LangSmith 付费	LlamaCloud	Cloud+自托管	企业平台	微软内部	开源免费

4.3 Agent 框架详细对比

框架	发布时间	核心机制	多 Agent	状态管理	与 LangChain 关系
LangGraph	2024.01	图编排 (StateGraph)	✅	内建持久化	LangChain 生态核心
Dify Workflow	2024	拖拽 DAG + 代码节点	✅	可视化	独立
CrewAI	2024	角色扮演多 Agent	✅	有限	**独立于 LC**
MAF (原SK)	2023→2026	Plugin + 流程框架	✅	.NET 生态	独立
LlamaIndex Workflows	2025	事件驱动 Agent	发展中	有限	独立
AutoGen	2023.08	多 Agent 对话	✅	有限	**维护模式**

4.4 企业能力对比

维度	LangChain	LlamaIndex	Dify	Haystack	MAF
可观测性	✅ LangSmith	❌ 弱	✅ 内置	✅ Enterprise	✅ .NET
可审计性	❌ 有限	❌ 有限	❌ 有限	✅ **核心优势**	✅ Enterprise
私有部署	✅ 企业版	✅ LlamaCloud	✅ 企业版	✅ Enterprise	✅ Azure
MCP 支持	发展中	发展中	✅ 原生	✅ Hayhooks	发展中
治理/合规	❌	❌	❌	✅	✅ M365 Copilot

4.5 用户口碑

LangChain：社区最大，文档最全，但"过度抽象"的抱怨一直没停过。"用 LCEL 写一个简单的问答都要 3 个 class，有时候直接调 API 更省事"（Reddit）。LangGraph 的 Agent 能力被认为是不可替代的优势。

LlamaIndex：检索精度和文档处理能力受认可。"用 LlamaIndex 处理 PDF 比 LangChain 的 Document Loader 好一个数量级"。但 Agent 能力不如 LangChain。

Dify：非技术团队的最爱。"我是产品经理，用 Dify 搭了个内部知识库机器人，没写过一行代码。" 但也有开发者抱怨："定制化需求深入一点就发现可视化节点的灵活性不够。"

Haystack：受监管行业的信仰之选。"在欧洲银行做 LLM 应用，Haystack 的可审计管线是合规团队唯一接受的方案。" 但社区活跃度不如 LangChain 和 Dify。

DSPy：评价两极分化。"用了 DSPy 后我们再也不用自己写 prompt 了" vs "学习曲线太陡，我们的团队适应不了。"

4.6 框架成本

平台	免费版	付费版
LangSmith	5k traces/月	Plus $39/席位/月
LlamaCloud	400K 信用点/月	Starter/Enterprise 定价
Dify Cloud	200 消息额度	Pro $59/月, Team $159/月
Haystack	开源免费	Enterprise 定制报价
MAF	开源免费	Azure 费用

所有框架本身开源，付费版本主要提供云服务、可观测性、企业级支持。

五、横纵交汇洞察

5.1 Dify 超越 LangChain 意味着什么

2026 年最引人注目的格局变化是 Dify 在 GitHub Stars 上超越 LangChain。这个数字落在"低代码"而非"编排"的框架上，揭示了一个深层趋势：

AI 应用的决策者和使用者正在从"工程师"扩展到"业务人员"。

LangChain 的 135k stars 来自 AI 工程师——他们需要编排能力来构建复杂的 Agent 系统。Dify 的 140k stars 来自更广泛的群体——产品经理、业务分析师、GCC 开发者——他们需要的是"不给开发团队增加负担就能做出 AI 应用"的工具。

这不是"低代码取代代码"的故事，而是**AI 应用的需求面在迅速扩大**——能用 Python 写 LangChain 的人就那么多，但需要一个内部知识库机器人的团队有几千倍那么多。

5.2 LangChain vs LlamaIndex：分工的建立和模糊

纵向历史中最清晰的模式是 LangChain 和 LlamaIndex 从"重叠"到"分工"的演变。

2022 年末他们几乎是同时出现的，定位有大量重叠（都是 Python 框架、都做 RAG、都做 prompt 管理）。市场上出现了大量 "LangChain vs LlamaIndex" 的对比文章。

但到了 2024–2025 年，行业找到了自然的分工：

- **做检索、索引、文档处理** → 用 LlamaIndex（LlamaParse 的文档解析能力无可替代）
- **做编排、Agent、工具调用** → 用 LangChain/LangGraph

2026 年的状态是"组合式堆栈"——两个框架的兼容性持续改善，实践中常组合使用。LlamaIndex 处理数据后输出结构化的上下文，LangGraph 编排 Agent 流程。

5.3 Agent 框架的"收敛还是不收敛"

2026 年，Agent 框架出现了一个有趣的分化：

- LangGraph 走"复杂编排"路线，面向需要深度定制的 Agent 系统
- CrewAI 走"简单易用"路线，面向快速原型
- Dify Workflow 走"可视化"路线，面向非技术人员

三者面临的不是同一个市场——它们是同一需求的不同抽象层次。 正如编程语言有底层（C）、中层（Java/Python）、高层（低代码平台）的层次，Agent 框架也在形成类似的分层：

抽象层次	典型框架	用户群体
低级（细粒度控制）	LangGraph	AI 工程师
中级（声明式）	CrewAI	全栈开发者
高级（可视化）	Dify Workflow	业务人员

5.4 微软的"主动收敛"和它的启示

Microsoft 在 2025–2026 年的双框架合并（Semantic Kernel → MAF，AutoGen → 维护模式）是一个有力的对照案例——**当一个公司内部有两个竞争框架时，最可能的结果不是共赢，而是收敛。**

MAF 的定位明确：服务于 .NET 生态和 M365 Copilot 治理。它不打算跟 LangChain 比"谁集成的 LLM 多"，而是绑定 Azure/AI Search/M365——通过生态锁定创造差异化优势。

这给了其他框架一个重要启示：**在充分竞争的市场中，"全能"不如"有钉子的生态位"。** Haystack 选择了审计，MAF 选择了 .NET 企业生态，Dify 选择了视化 + 自托管——走生态位而非全能的路线，反而获得了稳定的增长。

5.5 三个未来剧本

剧本一：Agent 框架收敛到 2–3 个主流选择（概率 60%）

框架的分工更加明确。LangGraph 成为 Agent 编排的事实标准（类似 Web 框架中的 React），Dify 成为低代码/L 平台的标准。LlamaIndex 在文档处理领域保持领先但 Agent 能力被 LangGraph 侵蚀。Haystack 守住监管行业。"每个生态位一个赢家"的格局基本定型。

剧本二：MCP 协议使框架层变薄，MCP 兼容性成为核心竞争力（概率 25%）

如果 MCP（Model Context Protocol）继续普及，框架的"集成连接器"价值被稀释——工具和数据源的连接从"框架专用接口"变成了"标准接口"。框架竞争力转移到"工作流编排质量"和"运行时效率"。Dify 率先原生支持 MCP 可能成为其长期优势。

剧本三：Agent 框架被 Agent 平台替代（概率 15%）

随着 AI Agent 的成熟，抽象的"框架"被更具体的"Agent 平台"替代——开发者不再用 LangGraph 代码搭建 Agent，而是用 Agent 平台配置 Agent（定义工具、分配模型、设定行为边界）。框架退居"Agent 平台的基础技术提供商"位置。

5.6 第一性原理追问

LLM 应用框架的终极问题是：**当 LLM 能力足够强时，框架还有必要吗？**

如果未来的 LLM 有 10M token 上下文、原生支持工具调用、内置记忆管理和规划能力——那么今天的框架封装的"让 LLM 能用工具、能检索、能记忆"的功能，会不会变得冗余？

2026 年的答案是：**不会完全消失，但框架的形态会变。** 今天的框架是"给 LLM 补能力"（模型做不到的框架来补），未来的框架是"给 LLM 做治理"（模型能做了，但怎么控制它、审计它、管理它——框架来做）。

这跟数据库的历史类似——SQL 查询优化器越来越强，但 ORM 框架没有消失，而是从"帮你写 SQL"变成了"帮你管理数据模型和迁移"。**框架的演进方向是从"增强能力"到"管理复杂度"。**

六、信息来源

框架官方来源

来源	URL	访问时间
LangChain GitHub	github.com/langchain-ai/langchain	2026-04-30
LangGraph 博客	blog.langchain.dev	2026-04-30
LlamaIndex GitHub	github.com/run-llama/llama_index	2026-04-30
LlamaIndex 网站	llamaindex.ai	2026-04-30
Dify GitHub	github.com/langgenius/dify	2026-04-30
Dify 官网	dify.ai	2026-04-30
Haystack GitHub	github.com/deepset-ai/haystack	2026-04-30
deepset/Haystack 关于页	deepset.ai/about	2026-04-30
Semantic Kernel GitHub	github.com/microsoft/semantic-kernel	2026-04-30
DSPy GitHub	github.com/stanfordnlp/dspy	2026-04-30
CrewAI GitHub	github.com/crewAIinc/crewAI	2026-04-30
AutoGen GitHub	github.com/microsoft/autogen	2026-04-30

其他来源

来源	URL	访问时间
Wikipedia LangChain 词条	wikipedia.org	2026-04-30
Forbes AI 50 (2025)	forbes.com	2026-04-30

本文是横纵分析系列的第 17 篇报告。研究方法论参考了由数字生命卡兹克（Khazix）提出的横纵分析法，融合语言学历时–共时分析、社会科学纵向–横截面研究设计、以及竞争战略分析的核心思想。